

Tema 2 — Modelado de Amenazas para la IA Agéntica

Índice del tema

2.1 El threat modeling adopta una mentalidad adversarial

2.2 Los frameworks tradicionales no alcanzan para la IA Agéntica

2.2.1 STRIDE, DREAD, PASTA, OCTAVE, LINDDUN (resumen) · 2.2.2 Sus limitaciones para la IA Agéntica · 2.2.3 IA Agéntica: un cambio de paradigma

2.3 La IA Agéntica introduce nuevos activos y superficies de ataque

2.3.1 Activos únicos · 2.3.2 Superficies de ataque únicas · 2.3.3 Recursos de referencia en la industria (2026)

2.4 MAESTRO: framework estructurado para la IA Agéntica

2.4.1 Un enfoque por capas · 2.4.2 Las siete capas · 2.4.3 Exploración detallada

2.4.4 Amenazas cross-layer · 2.4.5 El proceso de threat modeling de MAESTRO

2.5 Caso de estudio: Enterprise Copilot

2.6 Buenas prácticas y recomendaciones · 2.7 Conclusión

MAESTRO: un framework nuevo para amenazas de la IA Agéntica

- El tema presenta **MAESTRO** (Multi-Agent Environment, Security, Threat, Risk, and Outcome)
- Diseñado específicamente para el threat modeling de sistemas de **IA Agéntica**
- Recorrido: define la IA Agéntica y sus drivers técnicos, contrasta desafíos únicos vs. software tradicional
- Examina críticamente **STRIDE, DREAD, PASTA, OCTAVE, LINDDUN** frente a propiedades emergentes (no-determinismo, autonomía, identidad de agente, comunicación agente-a-agente)
- Presenta MAESTRO como framework de **siete capas**, con ejemplos concretos y foco en **cross-layer threats**
- Cierra con caso de estudio de un Enterprise Copilot y comparación con frameworks basados en listas

Keywords: MAESTRO framework · IA Agéntica · Threat modeling · Cross-layer threats · Foundation models · Agent autonomy · RAG pipeline · Prompt injection

La naturaleza autogobernada de los agentes exige threat modeling nuevo

- Los agentes inteligentes tienen naturaleza autogobernada, procesos de decisión complejos y comportamientos emergentes
- Esto exige un enfoque de threat modeling nuevo y más sofisticado
- **MAESTRO** (Huang, 2025): metodología novedosa y por capas
- Diseñada para diseccionar vulnerabilidades y superficies de ataque únicas de la IA Agéntica
- Ofrece una hoja de ruta integral para asegurar estas tecnologías

2.1 El threat modeling adopta una mentalidad adversarial

- **Threat modeling:** práctica fundamental de ciberseguridad — enfoque proactivo y estructurado para identificar, analizar y mitigar vulnerabilidades
- No es una casilla de cumplimiento ni un ejercicio posterior al desarrollo
- Parte integral del **secure software development lifecycle (SDLC)** y del diseño de sistemas
- Principio central: **mentalidad adversarial** — pensar como un atacante para anticipar el compromiso del sistema
- Permite "*build security in*" en vez de agregarla después (más costoso, complejo, menos efectivo)
- Objetivo: un sistema **resiliente por diseño**

2.1 Por qué es cada vez más crítico

- Sistemas más complejos, interconectados y distribuidos → superficie de ataque mayor y más diversa
- **Cloud computing**, microservicios e **IoT** amplificaron estos desafíos
- La emergencia de **AI/ML**, particularmente la **IA Agéntica**, introduce dimensiones de riesgo enteramente nuevas
- Los frameworks tradicionales de threat modeling están **mal equipados** para manejarlas

2.1 Beneficios de un threat modeling bien ejecutado (I)

- **Identificación temprana de vulnerabilidades:** idealmente en fase de diseño — mucho menos costoso que corregir tras el despliegue
- **Priorización de riesgos:** no todas las vulnerabilidades pesan igual; se evalúa probabilidad e impacto para enfocar esfuerzos
- Permite integrar medidas correctivas sin rework costoso

2.1 Beneficios de un threat modeling bien ejecutado (II)

- **Comunicación y colaboración mejoradas** entre desarrolladores, ingenieros de seguridad, arquitectos y negocio
- **Guía para inversiones en seguridad:** dónde invertir en controles, tecnologías y procesos
- **Soporte al cumplimiento regulatorio:** GDPR, HIPAA, PCI DSS, ISO 27001
- Ayuda a definir **requisitos de seguridad** verificables automáticamente
- En síntesis: gestión de riesgo continua e iterativa, no un evento único

2.2 Los frameworks tradicionales no alcanzan para la IA Agéntica

- El threat modeling evolucionó por décadas con distintos frameworks y metodologías
- Han demostrado su valor para software convencional
- Exhiben **limitaciones significativas** en el dominio de rápida evolución de la IA Agéntica
- Entender estas limitaciones justifica un enfoque nuevo: **MAESTRO**

2.2.1 STRIDE, DREAD, PASTA, OCTAVE, LINDDUN (resumen)

Framework	Origen	Foco
STRIDE	Microsoft (IriusRisk, 2023)	6 categorías (Spoofing, Tampering, Repudiation, Info Disclosure, DoS, Elevation of Privilege); se combina con DFDs
DREAD	Pure Storage, 2024	Scoring cualitativo (Damage, Reproducibility, Exploitability, Affected Users, Discoverability) para priorizar
PASTA	VerSprite, 2024	7 etapas centradas en riesgo y perspectiva del atacante
OCTAVE	Carnegie Mellon, 1999	Riesgo organizacional auto-dirigido, 3 fases; apto para grandes organizaciones
LINDDUN	Wuyts & Joosen, 2015	Amenazas de privacidad (Linkability, Identifiability, Non-repudiation, Detectability, Disclosure, Unawareness, Non-compliance)

Objetivo común: identificar y mitigar amenazas de forma estructurada — pero, como se ve a continuación, ninguno fue diseñado para las propiedades emergentes de la IA Agéntica.

2.2.2 Cinco limitaciones frente a la IA Agéntica

Los frameworks tradicionales encuentran limitaciones significativas frente a las características únicas de la IA Agéntica, derivadas de diferencias fundamentales en:

- **Determinismo**
- **Control**
- **Identidad**
- **Comunicación**
- **Adaptabilidad**

2.2.2 Limitación: Non-Determinism

- STRIDE y PASTA asumen flujo predecible: entradas → salidas determinísticas, acciones pre-programadas
- La IA Agéntica (especialmente con **LLMs**) exhibe **no-determinismo inherente**: el mismo prompt puede producir salidas distintas en momentos distintos, aun sin cambios en código/configuración
- Surge de la naturaleza probabilística de los LLMs y sus representaciones internas
- Los frameworks tradicionales no anticipan comportamiento emergente en interacciones complejas agente-entorno
- Requiere foco dedicado en integridad del modelo, alineación y ética (Huang et al., 2024)

2.2.2 Limitación: Autonomy

- Frameworks tradicionales asumen **control centralizado** (STRIDE se enfoca en componentes/flujos, no en decisión independiente)
- IA Agéntica: agentes autónomos con distintos grados de independencia
 - Agente de resumen con autonomía limitada → puede sesgar información sutilmente
 - Agente de seguridad semi-autónomo → puede crear excepciones a reglas por su propio juicio
 - Agente de compras totalmente autónomo → puede desviar recursos o comprometer procura
- No capturan cómo la autonomía afecta severidad ni el **goal misalignment**

2.2.2 Limitación: Agent Identity Management

- Frameworks tradicionales asumen identidades relativamente estáticas y jerarquía fija de privilegios
- IA Agéntica: identidades de agentes dinámicas — se crean/destruyen, roles y confianza cambian con el tiempo
- Sin mecanismos para modelar: **impersonación de agentes, escalación de privilegios entre agentes, delegación de autoridad entre agentes**

2.2.2 Limitación: Agent-to-Agent Communication

- STRIDE considera seguridad del canal (eavesdropping, tampering) pero como punto único de vulnerabilidad
- No modela el **contenido semántico** de la comunicación entre agentes
- La IA Agéntica multiagente depende fuertemente de intercambiar información, coordinar y negociar metas
- Sin mecanismos para: agente engañado por información falsa de otro agente, inyección de mensajes maliciosos, **colusión** entre agentes

2.2.2 Limitación: Lack of AI-Specific Threats

- Los frameworks tradicionales carecen de guía específica sobre **adversarial machine learning**
- No cubren: *data poisoning, evasion attacks, model extraction*

2.2.3 IA Agéntica: un cambio de paradigma

- La IA Agéntica representa un **cambio de paradigma fundamental** en desarrollo de software y diseño de sistemas
- No-determinismo de LLMs + autonomía + identidad de agentes + comunicación agente-a-agente → nuevos vectores de ataque y riesgos amplificados
- Huang publicó MAESTRO como blog post en **Cloud Security Alliance** (Huang, 2025)
- Este tema incorpora feedback de la comunidad cyber y extiende el framework

Tabla 2.1 — Software tradicional vs. IA Agéntica

Característica	Software tradicional	IA Agéntica
Determinismo	Mayormente determinista, comportamiento predecible	Inherentemente no-determinista, comportamiento emergente
Control	Centralizado, acciones pre-programadas	Descentralizado, toma de decisiones autónoma
Identidad	Principalmente basada en usuarios, identidades estáticas	Agentes como identidades independientes, relaciones dinámicas
Comunicación	Principalmente sistema-a-sistema o usuario-a-sistema	Agente-a-agente, interacciones y negociaciones complejas
Adaptabilidad	Limitada, requiere cambios de código	Aprendizaje y adaptación continuos
Complejidad	Relativamente predecible, manejable	Comportamiento emergente, difícil de predecir

2.3 La IA Agéntica introduce nuevos activos y superficies de ataque

- El threat modeling efectivo debe ir más allá del entendimiento tradicional de activos y superficies
- Los nuevos elementos se relacionan directamente con: **autonomía**, dependencia de modelos de IA, capacidad de **comunicarse e interactuar**, capacidad de **aprender y adaptarse**

2.3.1 Activos únicos — Foundation Models y Agent Memory

- **Foundation Models:** LLMs pre-entrenados u otros modelos de IA que sustentan razonamiento y decisión — propiedad intelectual significativa
 - Blanco de: *model extraction*, *poisoning*, ataques adversariales
- **Agent Memory** (corto y largo plazo): interacciones pasadas, información aprendida, estado interno
 - Blanco de: envenenamiento (info falsa), extracción de datos sensibles, manipulación del comportamiento

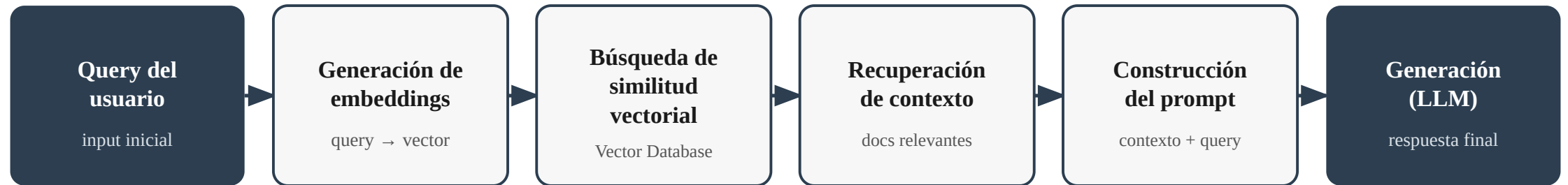
2.3.1 Activos únicos — Goals, Plans, Relationships

- **Agent Goals and Plans:** metas y estrategias del agente definen su comportamiento
 - Atacantes pueden manipularlas → "**goal hijacking**" / "**intent breaking**"
- **Agent Relationships and Trust Networks:** en sistemas multiagente, relaciones y confianza determinan interacción y coordinación
 - Explotables: impersonar agentes confiables, disrumpir comunicación, manipular la red de confianza

2.3.1 Activos únicos — Agent-Specific Data, RAG Pipeline, Vector DB

- **Agent-Specific Data:** logs de interacción, métricas, feedback y conocimiento generado por el propio agente — blanco para insights sobre su comportamiento
- **RAG Pipeline:** flujo end-to-end que enriquece respuestas con conocimiento externo (ver diagrama siguiente)
- **Vector Database:** almacena texto como vectores semánticos, habilita búsqueda por similitud (no por palabras clave)

Fig. — Pipeline RAG de extremo a extremo



Cada componente pasa su salida al siguiente: un sistema fluido que enriquece las respuestas de IA con conocimiento relevante de fuentes externas

2.3.2 Superficies de ataque — Input Space, Output Space

- **Input Space (Prompt Injection):** el input en lenguaje natural crea superficie vasta y compleja
 - Prompts maliciosos manipulan comportamiento, sortean controles, extraen información
 - Directa e indirecta — cubierto en **OWASP Top 10 for LLM Applications** (Huang et al., 2024)
- **Output Space (Hallucinations, Misinformation):** no-determinismo → salidas incorrectas o sin sentido
 - El propio agente se vuelve vector de desinformación/daño

2.3.2 Superficies de ataque — Comunicación y herramientas

- **Agent-to-Agent Communication Channels:** directos (mensajes) o indirectos (memoria/base compartida)
 - Riesgos: eavesdropping, tampering, inyección de mensajes, interrupción
- **Tool Integration Points:** APIs, bases de datos, servicios web conectados
 - Ejemplo: prompt injection para hacer ejecutar comandos maliciosos vía una herramienta conectada

2.3.2 Superficies de ataque — Frameworks y observabilidad

- **Agent Framework Vulnerabilities:** LangChain, AutoGen, Semantic Kernel, Anthropic MCP pueden tener vulnerabilidades propias en código, dependencias o configuración
- **Evaluation and Observability Systems:** atacantes pueden manipular métricas para ocultar actividad maliciosa, evadir detección de anomalías, o envenenar datos de evaluación

2.3.2 Superficies de ataque — Human-in-the-Loop y RAG

- **The Human-in-the-Loop:** la supervisión humana también puede ser superficie de ataque
 - Explotar sesgos cognitivos, **decision fatigue**, interacciones engañosas
- **RAG Pipeline and Vector DB:** el RAG introduce sus propias nuevas superficies de ataque
- Estos activos y superficies únicos justifican un enfoque de threat modeling específico: **MAESTRO**

2.3.3 Recursos de referencia en la industria (2026)

Contenido agregado por la cátedra — no forma parte del libro original (Huang & Hughes, 2025)

- Los activos y superficies que acabamos de ver no quedaron solo en la teoría: entre fines de 2025 y 2026 la industria formalizó frameworks públicos que los catalogan y priorizan
- Antes de entrar al detalle de MAESTRO, vale la pena situarlo junto a sus tres referencias más citadas hoy:
 - **OWASP GenAI LLM Top 10 (2026)** — riesgos de aplicaciones LLM
 - **OWASP Top 10 for Agentic Applications (2026)** — riesgos específicos de agentes autónomos
 - **MITRE ATLAS** — catálogo táctico de técnicas de ataque observadas contra sistemas de IA/ML
- No compiten con MAESTRO: operan en niveles de granularidad distintos y se referencian entre sí

OWASP GenAI LLM Top 10 (2026)

<https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/>

- Publicado el 4 de agosto de 2026 por el OWASP GenAI Security Project; actualiza las ediciones 2023/24 y 2025 con datos de miles de incidentes reales
- 10 categorías de riesgo (LLM01–LLM10:2026):
 - LLM01 Prompt Injection · LLM02 Sensitive Information Disclosure · LLM03 Excessive Agency · LLM04 Supply Chain · LLM05 Data and Model Poisoning
 - LLM06 Unbounded Consumption · LLM07 Misinformation · LLM08 Hidden Context Exposure · LLM09 Vector and Embedding Weaknesses · LLM10 Improper Output Handling
- Cambio notable vs. ediciones previas: **Excessive Agency** subió del puesto 6 al 3 — la propia OWASP marca la autonomía como el riesgo que más rápido crece
- Vincula explícitamente cada riesgo con NIST, CWE y **MITRE ATLAS**

OWASP Top 10 for Agentic Applications (2026)

<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>

- Publicado el 9 de diciembre de 2025, con más de 100 expertos de la industria. **Extiende** (no reemplaza) el LLM Top 10: los sistemas agénticos heredan esos riesgos y suman los propios
- 10 categorías de riesgo (ASI01–ASI10:2026, "Agentic Security Initiative"):
 - ASI01 Agent Goal Hijack · ASI02 Tool Misuse & Exploitation · ASI03 Agent Identity & Privilege Abuse · ASI04 Agentic Supply Chain Compromise · ASI05 Unexpected Code Execution
 - ASI06 Memory & Context Poisoning · ASI07 Insecure Inter-Agent Communication · ASI08 Cascading Agent Failures · ASI09 Human-Agent Trust Exploitation · ASI10 Rogue Agents
- Cada categoría está atada a un incidente real documentado (p. ej. ASI10 → el incidente de base de datos de producción de Replit)
- **No confundir** con la *OWASP Multi-Agent System Threat Modeling Guide* (2025) ya citada en este tema — son dos publicaciones distintas del mismo proyecto OWASP GenAI

MITRE ATLAS — catálogo táctico de ataques a IA/ML

<https://atlas.mitre.org/>

- *Adversarial Threat Landscape for Artificial-Intelligence Systems* — modelado explícitamente sobre **MITRE ATT&CK**, pero para sistemas de IA/ML
- Versión 5.x (2026): **16 tácticas, 84 técnicas, 56 sub-técnicas, 32 mitigaciones, 42 casos de estudio reales**
- Incorpora tácticas propias de IA — *AI Model Access, AI Attack Staging* — y desde noviembre de 2025 una táctica de *Command and Control*
- Octubre 2025 (colaboración con Zenity Labs): 14 técnicas nuevas específicas de agentes — envenenamiento de memoria/contexto, manipulación de configuración del agente, exfiltración vía invocación de herramientas
- Sitio **interactivo**: matriz navegable + casos reales documentados — conviene mostrarlo en vivo en el navegador durante la clase en vez de una captura estática

Cómo se complementan estos recursos con MAESTRO

Recurso	Formato	Responde a
OWASP LLM Top 10	Lista plana priorizada	¿Qué priorizar en el modelo/aplicación LLM?
OWASP Agentic Top 10	Lista plana priorizada	¿Qué priorizar en el agente autónomo?
MITRE ATLAS	Matriz táctica (estilo ATT&CK)	¿Cómo ataca un adversario, paso a paso?
MAESTRO	Arquitectura por capas	¿En qué capa vive el problema y cómo se propaga?

- No son alternativas entre sí: el propio OWASP LLM Top 10 2026 cita a MITRE ATLAS, y MAESTRO puede usar cualquiera de los tres como fuente de amenazas concretas para sus capas (Paso 2 del proceso MAESTRO, más adelante)
- A continuación, MAESTRO en detalle

2.4 MAESTRO: framework estructurado para la IA Agéntica

- Aborda las limitaciones de los frameworks tradicionales
- Considera explícitamente: **autonomía, no-determinismo, comunicación agente-a-agente, gestión de identidad**
- Provee una arquitectura **por capas** que facilita un análisis granular de amenazas

2.4.1 MAESTRO: un enfoque por capas (Granularity, Modularity)

- Basado en la **arquitectura de referencia de siete capas** para la IA Agéntica (Huang & Huang, 2025)
- **Granularity**: analiza amenazas en distintos niveles de abstracción, desde foundation models hasta el ecosistema de agentes — facilita ubicar el origen de una vulnerabilidad
- **Modularity**: cada capa se analiza independientemente; luego se consideran las interacciones cross-layer

2.4.1 MAESTRO: un enfoque por capas (cont.)

- **Comprehensiveness:** examinar sistemáticamente cada capa reduce el riesgo de pasar por alto vulnerabilidades — actúa como checklist
- **Scalability:** aplicable tanto a sistemas de un único agente como multiagente, sin importar complejidad
- Esta arquitectura se detalla en *Agentic AI: Theories and Practices* (Huang & Huang, 2025)

Fig. 2.2 — Las siete capas de MAESTRO

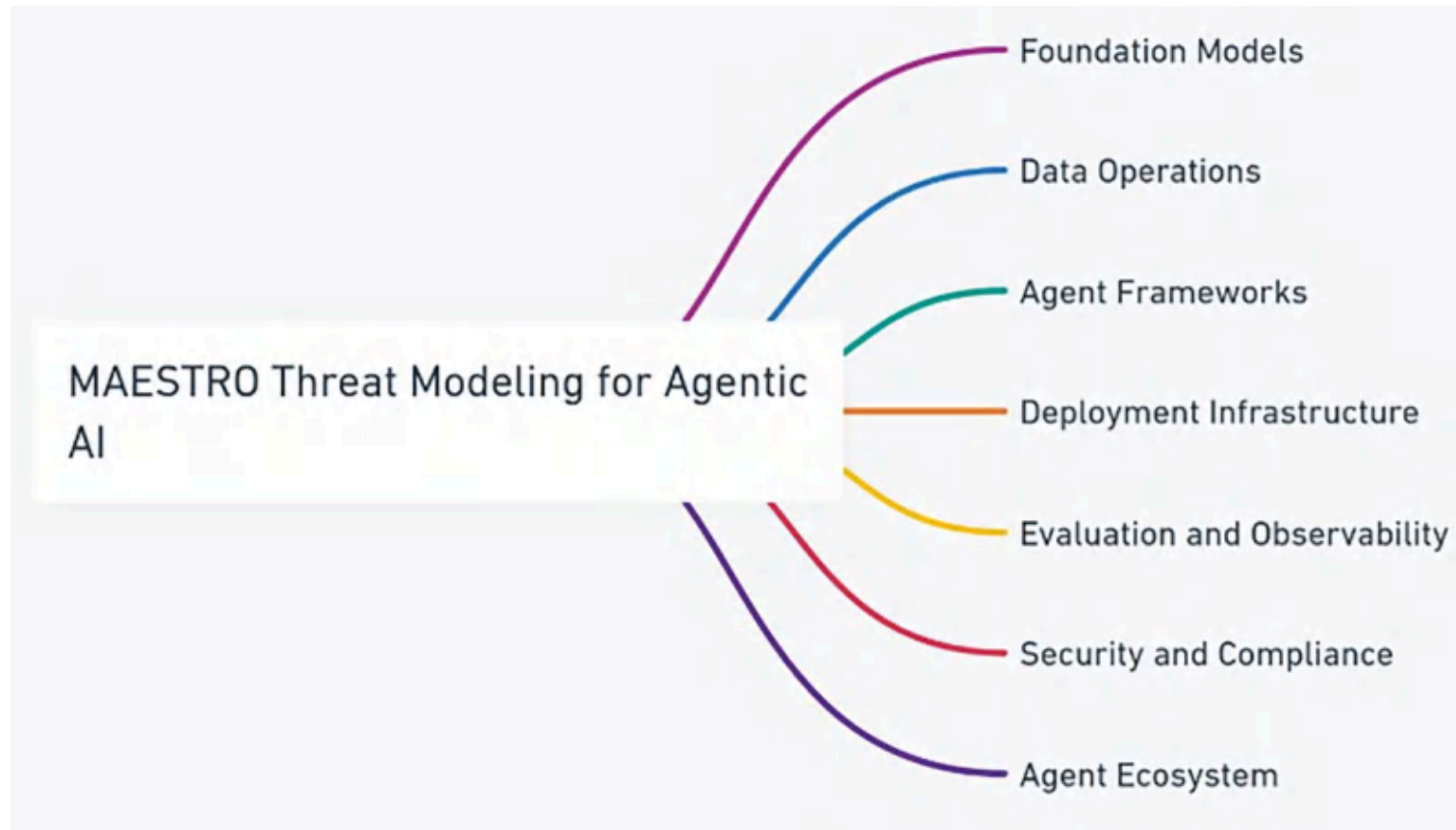


Fig. 2.2 MAESTRO: modelado de amenazas para la IA Agéntica — diagrama radial con las siete capas irradiando desde el centro: Foundation Models, Data Operations, Agent Frameworks, Deployment Infrastructure, Evaluation and Observability, Security and Compliance, Agent Ecosystem.

2.4.2 Las siete capas de MAESTRO — Capas 1 a 4

1. **Foundation Models:** LLMs u otros modelos de IA — el "cerebro" del agente; vulnerabilidades con consecuencias profundas para todo el sistema
2. **Data Operations** (énfasis en RAG y Vector Databases): recolección, almacenamiento, procesamiento y recuperación de datos
3. **Agent Frameworks:** LangChain, AutoGen, Semantic Kernel, Anthropic MCP — gestión de prompts, herramientas, memoria y comunicación
4. **Deployment Infrastructure:** cloud (AWS, Azure, GCP), contenedores (Docker, Kubernetes), redes y sistemas operativos

2.4.2 Las siete capas de MAESTRO — Capas 5 a 7

- 5. **Evaluation and Observability:** logging, métricas, detección de anomalías, explicabilidad
- 6. **Security and Compliance:** capa **vertical** que atraviesa todas las demás — autenticación, autorización, cifrado, DLP, cumplimiento (GDPR, HIPAA, CCPA)
- 7. **Agent Ecosystem:** entorno más amplio — otros agentes, usuarios humanos, servicios externos; registros y marketplaces de agentes

2.4.3 Exploración detallada de cada capa

- El tema recorre cada capa con amenazas concretas para un **AI assistant enfocado en RAG**
- Cada amenaza se etiqueta con el **factor agéntico** ("Agentic Factor") que explota:
- **Non-Determinism**
- **Autonomy**
- **Agent Identity Management**
- **Agent-to-Agent Communication**

(1) Foundation Models — amenazas (I)

- **Inyección de Prompt:** manipula el output del LLM, sorteando seguridad, extrae información — *"Ignore previous instructions..."* → **Non-Determinism**
- **Jailbreak del Modelo:** sorteando mecanismos de seguridad para generar contenido dañino → **Non-Determinism**
- **Inversión/Extracción del Modelo:** consultas repetidas reconstruyen datos de entrenamiento o roban el modelo → **Non-Determinism**

(1) Foundation Models — amenazas (II)

- **Envenenamiento de Datos (Fase de Entrenamiento):** datos maliciosos en el entrenamiento introducen sesgo/backdoor → **Non-Determinism**
- **Inestabilidad del Modelo:** outputs inconsistentes ante inputs similares → **Non-Determinism**
- **Ataques de Puerta Trasera (Backdoor):** triggers ocultos disparan comportamiento específico → **Non-Determinism**
- **Modelo vulnerable a jailbreak:** el modelo puede ser jailbreakeado con éxito

(2) Data Operations — amenazas (I)

- **Envenenamiento de Datos (RAG):** documentos maliciosos inyectados en las fuentes de RAG → info incorrecta/dañina (N/A — integridad de datos)
- **Envenenamiento de Embeddings:** embeddings maliciosos en la vector DB → recuperación de contenido controlado por el atacante → **Agent-to-Agent Communication** (indirecta)
- **Deriva Semántica:** embeddings desactualizados por cambio de significado → recuperación inexacta → **Non-Determinism**

(2) Data Operations — amenazas (II)

- **Validación de Entrada Insuficiente:** permite inyección de código/caracteres especiales (p. ej. SQL injection) — vulnerabilidad de manejo de datos
- **Controles de Acceso Débiles en la Base de Datos Vectorial:** agentes no autorizados acceden a embeddings sensibles → **Identity Management**
- **Falta de Rastreo de Procedencia:** no se puede rastrear el origen de la información recuperada — vulnerabilidad de gestión de datos

(2) Data Operations — amenazas (III)

- **Denegación de Servicio por Sobrecarga de la Base de Datos Vectorial:** alto volumen de consultas agota recursos
→ **Autonomy** (si lo inicia un agente comprometido)
- **Vulnerabilidad del Espacio de Embeddings Compartido:** consultas de un agente influyen en resultados de otros
→ **Agent-to-Agent Communication** (indirecta)
- **Dependencia Excesiva de la Información del RAG:** dependencia excesiva de información desactualizada o sesgada → **Autonomy**

(3) Agent Frameworks — amenazas (I)

- **Espacio de Acciones Demasiado Permisivo:** el framework permite un rango de acciones más amplio del necesario → **Autonomy**
- **Encadenamiento de Tareas No Intencionado:** el agente encadena tareas de forma imprevista e insegura → **Autonomy**
- **Comunicación Insegura Entre Agentes:** canal sin cifrar entre agentes → **Agent-to-Agent Communication + Identity Management** (sin autenticación)

(3) Agent Frameworks — amenazas (II)

- **Corrupción de Estado a Nivel del Framework:** bug del framework (no del LLM) corrompe estado interno → **Non-Determinism**
- **Agente que Excede sus Límites:** el agente toma acciones fuera de su rol definido → **Autonomy**
- **Credenciales/API Keys Hardcodeadas:** secretos almacenados de forma insegura en código/configuración — mala práctica
- **Ventana de Contexto sin Restricciones:** aumenta vulnerabilidad a prompt injection y degrada desempeño

(4) Deployment Infrastructure — amenazas (I)

- **Endpoints de API del Agente sin Protección:** sin autenticación/autorización adecuadas → **Identity Management**
- **Segmentación de Red Inadecuada:** agentes mal aislados entre sí y de otros componentes → **Agent-to-Agent Communication** no autorizada
- **Dependencias Vulnerables:** el framework depende de paquetes vulnerables

(4) Deployment Infrastructure — amenazas (II)

- **Agotamiento de Recursos:** inundación de requests deja el sistema no disponible — vulnerabilidad de infraestructura
- **Exposición de Cuenta de Servicio:** credenciales de cuenta de servicio expuestas (aunque no comprometidas)
- **Sistema de Orquestación Comprometido:** el sistema de orquestación de contenedores (p. ej. Kubernetes) es comprometido

(5) Evaluation and Observability — amenazas (I)

- **Falta de Auditoría en Interacciones Agente-a-Agente:** sin logs de comunicación entre agentes → **Agent-to-Agent Communication**
- **Ausencia de Detección de Anomalías:** comportamiento inusual no se marca como sospechoso → **Non-Determinism, Autonomy**
- **Sobrecarga del Sistema de Monitoreo:** demasiados agentes/eventos saturan el monitoreo → **Agent-to-Agent Communication**

(5) Evaluation and Observability — amenazas (II)

- **Evasión por Mimetismo de Acciones Benignas:** acciones maliciosas disfrazadas de comportamiento normal → **Autonomy**
- **Auto-desactivación del Logging por el Agente:** el agente desactiva autónomamente su propio logging
- **Envenenamiento de los Datos de Monitoreo:** datos falsos en las fuentes de monitoreo enmascaran actividad maliciosa → relacionado con **Autonomy**

(6) Security and Compliance (capa vertical) — amenazas (I)

- **Políticas IAM Débiles del Agente:** políticas IAM débiles/mal configuradas otorgan acceso más allá del previsto → **Identity Management**
- **Herencia de Permisos Entre Agentes:** un agente hereda permisos de otro sin control → escalación de privilegios → **Identity Management + Agent-to-Agent Communication**
- **Violaciones de Residencia de Datos:** datos procesados fuera de fronteras autorizadas — problema de cumplimiento

(6) Security and Compliance (capa vertical) — amenazas (II)

- **Falla en la Aplicación Dinámica de Políticas:** falla en el enforcement dinámico de políticas → **Non-determinism/Autonomy**
- **Falla en la Resolución de Conflictos de Políticas:** políticas en conflicto entre Copilots que interactúan → resultados impredecibles → **Agent-to-Agent Communication**
- **Control de Acceso Insuficiente en Fuente de Datos Conectada:** un agente accede a un recurso que no debería

(7) Agent Ecosystem — amenazas (I)

- **Registro de un Agente Rogue (Rogue Agent):** un actor malicioso registra un agente rogue en el registro → **Identity Management** (falta de vetting) + **Agent-to-Agent Communication**
- **Mecanismo de Descubrimiento de Agentes Inseguro:** DNS spoofing redirige agentes a un servidor malicioso → **Identity Management, Agent-to-Agent Communication**
- **Publicidad de Agente Malicioso:** capacidades falsas anunciadas en el marketplace → **Agent-to-Agent Communication**

(7) Agent Ecosystem — amenazas (II)

- **Agente descargado automáticamente del marketplace:** un agente descarga otros agentes automáticamente → **Autonomy**
- **Calificaciones de Agentes Inconsistentes o Poco Confiables:** sistema de reputación poco confiable o manipulable — no-determinismo del propio marketplace
- **Canal de Comunicación Comprometido dentro del Marketplace:** eavesdropping, modificación de mensajes, inyección de contenido → **Agent-to-Agent Communication**

(7) Agent Ecosystem — amenazas (III)

- **Actualizaciones de Agente sin Revisión de Seguridad (Vetting):** nuevas versiones sin vetting de seguridad pueden traer backdoors → **Autonomy + Identity Management**
- **Colusión Emergente:** agentes individualmente benignos desarrollan un patrón imprevisto que expone información sensible → **Agent-to-Agent Communication, Autonomy, Non-Determinism**

2.4.4 Cross-layer threats: el riesgo interconectado

- Aspecto crítico de MAESTRO: identificar amenazas **cross-layer** que abarcan múltiples capas
- Más complejas y difíciles de detectar que las de una sola capa, pero de **impacto mucho mayor**
- Pueden llevar a fallas en cascada o compromiso de todo el sistema
- El tema agrupa los ejemplos ya vistos en cinco categorías generales

Fig. — Matriz de cross-layer threats

Categoría	L1	L2	L3	L4	L5	L6	L7	Factores agénticos
I	●	○	●	○	○	○		Non-Determinism + Autonomy
II		○	●	○		○	●	Autonomy + Agent-to-Agent Comm.
III		○	●	○		●	○	Identity Mgmt + Autonomy
IV		○	●	○		●	●	Agent-to-Agent Comm. + Identity Mgmt
V		○	○	○	●	○	○	Observability + cualquier otra capa

● = capa clave (siempre involucrada) · ○ = capa frecuente (involucrada a menudo, según el caso)

Las categorías no son mutuamente excluyentes: la clave es analizar sistemáticamente las interacciones entre capas, no memorizar la tabla.

Cross-Layer: Categorías I y II

I. Non-Determinism + Autonomy (capas clave: 1, 3)

- Comportamiento impredecible del foundation model (Capa 1) + acciones autónomas del agente (Capa 3)
- Ejemplo: una alucinación del LLM lleva al agente a modificar autónomamente permisos de acceso (Capa 3/6) → brecha de datos

II. Autonomy + Agent-to-Agent Communication (capas clave: 3, 7)

- Ejemplo: un agente delega autónomamente una tarea a otro (Capa 3/7) sin autorización adecuada → filtración de datos (Capa 6)

Cross-Layer: Categorías III y IV

III. Identity Management + Autonomy (capas clave: 3, 6)

- Ejemplo: un atacante obtiene credenciales de un agente (Capa 6) y usa su autonomía (Capa 3) para acciones no autorizadas

IV. Agent-to-Agent Communication + Identity Management (capas clave: 3, 6, 7)

- Ejemplo: un agente rogue (Capa 7) impersona a un agente confiable (Capa 6) y manipula a otros vía comunicación agente-a-agente (Capa 3/7)

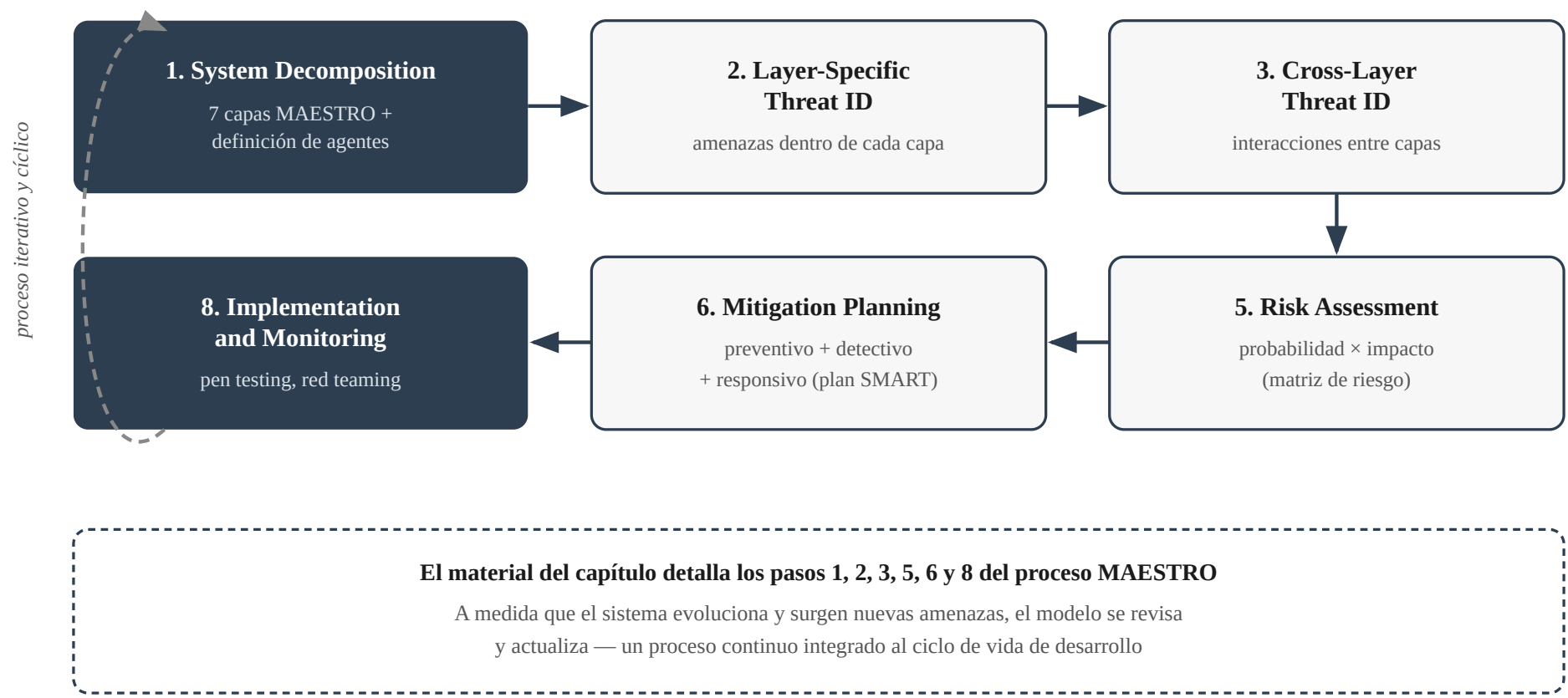
Cross-Layer: Categoría V y cierre

V. Observability (Capa 5) + otras capas (capa clave: 5, y cualquier otra donde se origine la amenaza)

- Vulnerabilidades en observabilidad dificultan detectar y mitigar amenazas de otras capas
- Ejemplo: un agente comprometido (Capa 3) evade la detección del sistema de monitoreo (Capa 5)

Las categorías no son mutuamente excluyentes: la clave es analizar sistemáticamente las interacciones entre capas.

Fig. — El proceso MAESTRO es iterativo y cíclico



2.4.5 El proceso MAESTRO consta de 6 pasos

- MAESTRO es un **proceso estructurado**, no solo una lista de amenazas — iterativo, integrado a todo el SDLC
- El diagrama del ciclo (slide anterior) define exactamente **6 fases**: System Decomposition, Layer-Specific Threat ID, Cross-Layer Threat ID, Risk Assessment, Mitigation Planning, Implementation and Monitoring
- *Nota de edición*: la fuente (Huang & Hughes, 2025) numera estos pasos como 1, 2, 3, 5, 6 y 8 — sin que exista contenido para los pasos 4 y 7 en ningún lugar del tema. Aquí se reenumeran consecutivamente **Paso 1 a Paso 6** para no sugerir pasos faltantes inexistentes

2.4.5 Paso 1 — System Decomposition

- Descomponer el sistema de IA Agéntica en sus componentes, siguiendo las siete capas de MAESTRO
- Identificar tecnologías específicas por capa: LLM usado (Capa 1), pipeline de RAG y tipo de vector DB — p. ej. Pinecone, Weaviate, Faiss (Capa 2), framework de agentes — p. ej. LangChain, AutoGen (Capa 3), etc.
- Da entendimiento claro de la arquitectura y la superficie de ataque por capa
- Crucial: también define agentes, sus capacidades, metas, fuentes de datos e interacciones

2.4.5 Paso 2 — Layer-Specific Threat Identification

- **Generic Threat Examples:** usar los ejemplos del tema como punto de partida
- **Specific Technologies:** investigar vulnerabilidades conocidas (CVE, avisos de proveedores, papers de investigación)
- **Agentic Factors:** ¿cómo podrían explotarse no-determinismo, autonomía, identidad y comunicación en cada capa?
- **Existing Threat Intelligence:** aprovechar fuentes de inteligencia de amenazas emergentes

2.4.5 Paso 3 — Cross-Layer Threat Identification

- Analizar interacciones y dependencias entre capas — aquí se manifiesta el verdadero poder de MAESTRO
- Preguntas guía: ¿cómo una vulnerabilidad en el Foundation Model (Capa 1) compromete el pipeline de RAG (Capa 2)? ¿cómo un agente comprometido (Capa 3) usa la infraestructura (Capa 4) para DoS?
- ¿cómo una debilidad de identidad (Capa 6) manipula la comunicación agente-a-agente (Capa 7)?
- Crucial para identificar fallas en cascada y escenarios de ataque complejos

2.4.5 Pasos 4 y 5 — Risk Assessment y Mitigation Planning

4. Risk Assessment: evaluar probabilidad de explotación e impacto — **matriz de riesgo** (bajo/medio/alto/crítico) para priorizar

5. Mitigation Planning: combinar tres tipos de controles

- **Preventative:** autenticación fuerte, validación de inputs, codificación segura, cifrado, segmentación de red
- **Detective:** logging, monitoreo, detección de intrusiones y anomalías, auditorías
- **Responsive:** respuesta a incidentes, recuperación de datos, remediación automatizada, failover
- El plan debe ser **SMART** y alinearse con la tolerancia al riesgo de la organización

2.4.5 Paso 6 — Implementation and Monitoring

- Implementar las mitigaciones elegidas y monitorear continuamente el sistema
- Proceso continuo, no actividad única: el panorama de amenazas evoluciona constantemente
- Requiere **penetration testing** periódico, **red teaming**, escaneos de vulnerabilidades
- El proceso MAESTRO es **iterativo y cíclico**: se revisa y actualiza a medida que el sistema evoluciona

2.5 Caso de estudio: Enterprise Copilot

- Escenario real: un **Enterprise Copilot** (similar a Microsoft Copilot) desplegado en una gran organización
- Integrado con email (Outlook), calendario, almacenamiento (SharePoint, OneDrive), CRM (Dynamics 365), Teams
- Tareas del Copilot: resumir emails/documentos, programar reuniones, recuperar información, generar reportes, automatizar flujos, dar recomendaciones contextuales

2.5 System Decomposition (Copilot) — capas 1 a 4

- **Layer 1:** LLM(s) específico(s) — p. ej. versión personalizada de GPT-4
- **Layer 2:** pipeline de RAG crucial — vector DB (Azure Cognitive Search o propia), fuentes (SharePoint, OneDrive, wikis, bases de datos), modelo de embeddings
- **Layer 3:** framework de agentes (interno de Microsoft + posibles librerías de terceros)
- **Layer 4:** infraestructura Azure (AKS, Azure Functions)

2.5 System Decomposition (Copilot) — capas 5 a 7

- **Layer 5:** herramientas internas + Azure Monitor (logging, métricas, anomalías)
- **Layer 6:** Azure Active Directory / EntraID, políticas de seguridad de Microsoft 365, reglas DLP, cumplimiento (GDPR, CCPA)
- **Layer 7:** interacción con otros Copilots, usuarios internos/externos, servicios de Microsoft 365

2.5 Threat Identification (Copilot)

- **Layer 1:** prompt injection dirigido al LLM
- **Layer 2:** semantic drift en embeddings de SharePoint
- **Layer 3:** unintended task chaining por fallas en la lógica del framework
- **Layer 4:** falla en rotación de claves de cuentas de servicio Azure
- **Layer 5:** evasión del monitoreo vía benign action mimicry
- **Layer 6:** filtración de datos cross-application vía portapapeles compartido
- **Layer 7:** plugin de Copilot rogue explota confianza en Microsoft 365
- **Cross-layer:** alucinaciones en cascada → permisos incorrectos de SharePoint; delegación mal alineada → filtración a Teams; escalación de privilegios vía calendario + Graph API

2.5 Risk Assessment (Copilot)

- **Prompt injection** contra el LLM (Capa 1): **alta probabilidad** (vulnerabilidad conocida) y **alto impacto** (brechas de datos, acciones no autorizadas)
- **Semantic drift** en embeddings de SharePoint (Capa 2): **probabilidad media** e **impacto medio** (recuperación incorrecta, no necesariamente brecha directa)

2.5 Mitigation Planning (Copilot)

- **Prompt Injection:** validación robusta de inputs, filtrado de outputs, entrenamiento adversarial
- **Data Poisoning:** rastreo de procedencia, validación de datos, detección de anomalías
- **Privilege Escalation:** mínimo privilegio, autenticación multifactor para cuentas de servicio, auditorías regulares
- **Unintended Task Chaining:** espacio de acción bien definido, manejo robusto de errores, verificación formal
- **Cross-Layer Threats:** combinación de controles a través de capas — p. ej. robustez del LLM (Capa 1) + controles de acceso más estrictos (Capa 6)

2.5 Implementation and Monitoring (Copilot)

- **Logging:** acciones del agente, recuperaciones de RAG, llamadas a API, interacciones de usuarios
- **Metrics Collection:** frecuencia de prompt injection, tasa de alucinaciones, accesos no autorizados, volumen de datos transferidos
- **Anomaly Detection:** ML o reglas para comportamiento inusual
- **Regular Security Audits:** auditorías periódicas y penetration testing

2.5 Cierre del caso de estudio

- El modelo de amenazas se revisa y actualiza regularmente a medida que el Copilot evoluciona
- Demuestra la aplicación práctica de MAESTRO: descomponer, identificar amenazas por capa y cross-layer, evaluar riesgo, planificar mitigaciones
- Para otros casos de uso: guía de **OWASP (2025)** sobre MAESTRO para Multi-Agent Systems

2.6 Buenas prácticas — principios generales

- **Defense in Depth:** múltiples capas de controles — crucial dado el no-determinismo
- **Principle of Least Privilege:** permisos mínimos necesarios para agentes y usuarios
- **Assume Breach:** diseñar asumiendo que el sistema será violado
- **Secure by Design:** seguridad integrada en cada etapa del SDLC
- **Continuous Monitoring and Improvement:** la seguridad no es un esfuerzo único

2.6 Recomendaciones específicas de MAESTRO

- **Adopt a Layered Approach:** analizar sistemáticamente cada nivel de abstracción
- **Prioritize Cross-Layer Threats:** las de mayor complejidad e impacto
- **Address Agentic-Specific Risks:** no-determinismo, autonomía, comunicación agente-a-agente, gestión de identidad
- **Embrace Continuous Threat Modeling:** proceso continuo, integrado al ciclo de vida de desarrollo

2.6 Mitigaciones específicas — Layer 1 (Foundation Models)

- Robust Input Validation and Output Filtering (modelos/servicios separados para bloquear outputs dañinos)
- Adversarial Training
- Model Monitoring (alucinaciones, sesgo, comportamiento indeseable)
- Jailbreak detection

2.6 Mitigaciones específicas — Layer 2 (Data Operations)

- Data Provenance Tracking
- Data Validation and Sanitization
- Access Control for Vector Databases
- Regular Embedding Updates (previene semantic drift)
- Embedding Monitoring
- Input Validation for RAG Pipeline

2.6 Mitigaciones específicas — Layer 3 (Agent Frameworks)

- Secure Coding Practices
- Principle of Least Privilege
- Action Space Limitation
- Robust Error Handling
- Secure Inter-Agent Communication (cifrado, autenticación mutua)
- Input Validation and Sanitization
- Context Limitation (reduce prompt injection y mejora desempeño)

2.6 Mitigaciones específicas — Layer 4 (Deployment Infrastructure)

- Secure API Endpoints
- Network Segmentation
- Regular Security Updates and Patching
- Resource Limits (CPU, memoria, ancho de banda)
- Infrastructure Monitoring
- Secure Orchestration System (pod security policies, network policies, gestión de secretos, RBAC)

2.6 Mitigaciones específicas — Layer 5 (Evaluation and Observability)

- Comprehensive Logging
- Anomaly Detection
- Performance Monitoring
- Explainability Techniques
- Regular Security Audits
- Data Integrity Verification (checksums, firmas digitales, hashes)

2.6 Mitigaciones específicas — Layer 6 (Security and Compliance)

- Strong Identity and Access Management (IAM)
- Data Loss Prevention (DLP)
- Compliance with Regulations
- Regular Security Assessments
- Dynamic Policy Enforcement (ajuste automático según riesgo, contexto, sensibilidad e inteligencia de amenazas)

2.6 Mitigaciones específicas — Layer 7 (Agent Ecosystem)

- Agent Vetting and Onboarding
- Reputation Systems
- Secure Communication Protocols
- Agent Isolation
- Ecosystem Monitoring
- Agent Registry Security (firmas digitales, control de versiones, auditoría a prueba de manipulación)

2.6 Capacitación y conciencia, y cierre

- **Developer Training:** prevenir prompt injection, interactuar con seguridad con LLMs/RAG, autenticación y autorización robustas
- **User Awareness:** riesgos de alucinaciones, desinformación, ingeniería social
- Enfoque de seguridad proactivo, por capas y centrado en el agente

2.7 Conclusión (I)

- MAESTRO ofrece una solución robusta e integral: arquitectura de siete capas para descomponer sistemas y analizar amenazas nivel por nivel
- A diferencia de las listas planas de amenazas, permite ubicar el origen de vulnerabilidades y diseñar mitigaciones focalizadas
- Foco explícito en **amenazas cross-layer**: previene fallas en cascada y compromiso de todo el sistema

2.7 Conclusión (II)

- La IA Agéntica se integra cada vez más profundamente en negocios y vidas cotidianas
- Un framework robusto como MAESTRO es esencial para un despliegue seguro y responsable
- Combina buenas prácticas tradicionales + controles específicos de IA + threat modeling continuo e iterativo
- El futuro de la IA depende de sistemas poderosos y seguros y confiables

Referencias del tema (I)

Carnegie Mellon University. (1999). *Operationally critical threat, asset, and vulnerability evaluation (OCTAVE)*. Software Engineering Institute.

Huang, K. (2025, February 6). *Agentic AI threat modeling framework: MAESTRO*. Cloud Security Alliance.

Huang, K., Goertzel, B., Wu, D., & Xie, A. (2024). GenAI model security. En *Generative AI security: Theories and practices* (pp. 163–198). Springer.

Huang, K., & Huang, J. (2025). AI agent tools and frameworks. En *Agentic AI: Theories and practices* (pp. 23–50). Springer.

Huang, K., Huang, G., Dawson, A., & Wu, D. (2024). GenAI application level security. En *Generative AI security: Theories and practices* (pp. 199–237). Springer.

Referencias del tema (II)

IriusRisk. (2023, September 29). *Threat modeling methodology: STRIDE*.

OWASP. (2025). *Multi-agentic system Threat Modeling Guide v1.0*.

Pure Storage. (2024, November 26). *What is the DREAD threat model?*

VerSprite. (2024, December 15). *Process for attack simulation and threat analysis (PASTA)*.

Wuyts, K., & Joosen, W. (2015). *LINDDUN: A privacy threat modeling framework*. KU Leuven.

Referencias del tema (III) — recursos agregados por la cátedra (2026)

No forman parte del libro original (Huang & Hughes, 2025) — corresponden a la nueva sección 2.3.3

MITRE. (2026). *MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems* (v5.x).

<https://atlas.mitre.org/>

OWASP GenAI Security Project. (2025, December 9). *OWASP Top 10 for Agentic Applications 2026*.

<https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/>

OWASP GenAI Security Project. (2026, August 4). *OWASP GenAI LLM Top 10 2026*.

<https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/>